

อันนี้เป็น document สำหรับ ขั้นตอนอย่างละเอียด ทำระบบตามรายละเอียดด้านล่าง ซึ่งเอาไว้ test

*สำหรับเปเอง (มีทั้งสร้าง ฐานข้อมูลจำลองขึ้นมาด้วย) (เพื่อผมลืม)

*ตั้งแต่หน้า 6-9 สาระ นอกนั้นสาหร่าย

***สิ่งที่อาจต้องแก้เพิ่ม**

-ชื่อตัวแปร ใน lambda function

-รูปแบบการส่ง response ใน lambda function

5. ระบบเช็ควันหมดอายุ + EventBridge

- Lambda สำหรับสแกน DynamoDB เพื่อดูวัตถุที่ใกล้หมดอายุ (node.js 20.x)
- EventBridge ตั้งเวลา (cron job)
- ไม่ต้องพึ่ง Frontend เลย

แยกทำได้อิสระจากระบบหลัก

DynamoDB

Table name: Ingredients

Partition key: id (string)

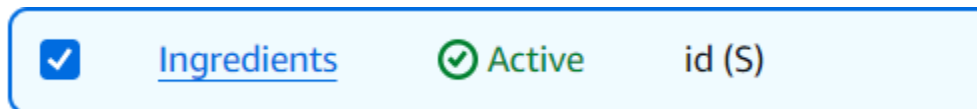
Sort key: ไม่กรอก

Table settings: Default settings

*****เพิ่ม test data*****

- DynamoDB -> Tables ->

กดเลือก Ingredients

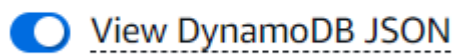


-Action -> Explore items -> Create item

-เลือก JSON view



-ติ๊กออก

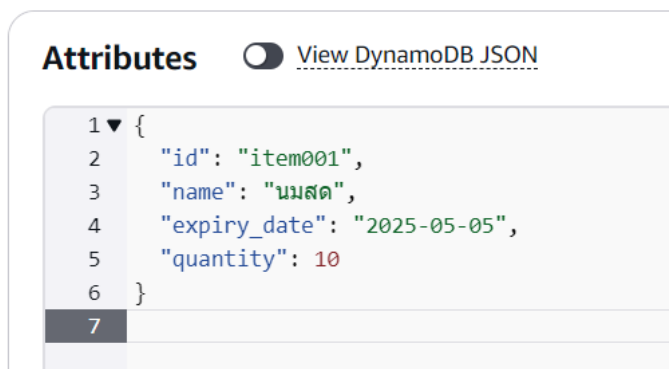


-edit ข้างในด้วยข้อมูลใหม่ หรือ item ใหม่ เช่น

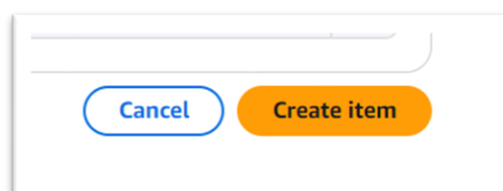
```
{  
  
  "id": "item001",  
  
  "name": "นมสด",  
  
  "expiry_date": "2025-05-05",  
  
  "quantity": 10  
}
```

หรือ

```
{  
  
  "id": "item002",  
  
  "name": "ไข่ไก่",  
  
  "expiry_date": "2025-05-08",  
  
  "quantity": 30  
}
```



-กด Create item สีส้ม



Amazon EventBridge

Name: CheckIngredientsDaily

Description: Daily Ingredient Check

Rule type | Info

☐ Rule with an event pattern
A rule that runs when an event matches the defined event pattern. EventBridge sends the event to the specified target.

☒ Schedule
A rule that runs on a schedule

- Step 1
Define rule detail
- Step 2
Define schedule
- Step 3
Select target(s)
- Step 4 - optional
Configure tags
- Step 5
Review and create

Define rule detail | Info

Rule detail

Name
CheckingredientsDaily
Maximum of 64 characters consisting of numbers, lower/upper case letters, -, _.

Description - optional
Daily Ingredient Check

Event bus | Info
Select the event bus this rule applies to, either the default event bus or a custom or partner event bus.
default

☒ Enable the rule on the selected event bus

Rule type | Info

☐ Rule with an event pattern
A rule that runs when an event matches the defined event pattern. EventBridge sends the event to the specified target.

☒ Schedule
A rule that runs on a schedule

EventBridge Scheduler - A new AWS scheduling capability! New

A new EventBridge scheduling functionality that provides one-time and recurring scheduling functionality independent of Event buses and rules. You can create a schedule to invoke targets such as a Lambda function.
[Learn More](#)

[Continue to create rule](#)

[Cancel](#)

[Continue in EventBridge Scheduler](#)

The screenshot shows the Amazon EventBridge console interface. The top navigation bar includes the AWS logo, a search bar, and the current region (us-east-1). The left sidebar contains the 'Amazon EventBridge' menu with options like Dashboard, Developer resources, Buses, Pipes, Scheduler, and Integration. The main content area is titled 'Create rule' and shows the 'Define schedule' step. The 'Schedule pattern' section offers two options: a fine-grained schedule (selected) and a regular rate schedule. The 'Cron expression' section shows a cron expression '0 1 * * ? *' with a dropdown for 'UTC'. Below this, a list of 'Next 10 trigger date(s)' is displayed. The bottom of the console shows the Windows taskbar with the date and time (11:33 PM 5/3/2025).

Launch AWS Academy Learner

Items | Amazon DynamoDB M...

CheckingExpirationDate | Func...

Log Groups | CloudWatch | us...

Create rule > Select target(s) | /

us-east-1.console.aws.amazon.com/events/home?region=us-east-1#/rules/create

Search [Alt+S]

United States (N. Virginia)

vodablu/user2924807-6609520116_kamomphan_kanthayot @ 4111-110...

Amazon EventBridge > Rules > Create rule

Amazon EventBridge

Dashboard [New](#)

Developer resources

Learn

Sandbox

Quick starts

Buses

Event buses

Rules

Global endpoints

Archives

Replays

Pipes

Pipes

Scheduler

Schedules

Schedule groups

Integration

Partner event sources

API destinations

Connections [Updated](#)

Schema registry

Step 4 - optional

Configure tags

Step 5

Review and create

Target 1

Target types

Select an EventBridge event bus, EventBridge API destination (SaaS partner), or another AWS service as a target.

☐ EventBridge event bus

☐ EventBridge API destination

☒ AWS service

Select a target [Info](#)

Select target(s) to invoke when an event matches your event pattern or when schedule is triggered (limit of 5 targets per rule)

Lambda function

Target location

☒ Target in this account

☐ Target in another AWS account

Function

CheckingExpirationDate

Configure version/alias

Permissions

☒ Use execution role (recommended)

Execution role

EventBridge needs permission to send events to the target specified above. By continuing, you are allowing us to do so. [EventBridge and AWS Identity and Access Management](#)

☐ Create a new role for this specific resource

☒ Use existing role

Role name

LabRole

Additional settings

cloudShell Feedback

© 2025, Amazon Web Services, Inc. or its affiliates. [Privacy](#) [Terms](#) [Cookie preferences](#)

28°C

Search

ENG

11:33 PM

5/3/2025

Lampda function

Name: CheckingExpirationDate

Runtime: Node.js 20.x

Runtime

Info

Choose the language to use to write your function. Note that the console code editor supports only Node.js, Python, and Ruby.

Node.js 20.x

LabRole

▼ Change default execution role

Execution role

Choose a role that defines the permissions of your function. To create a custom role, go to the [IAM console](#).

- ☐ Create a new role with basic Lambda permissions
- ☒ Use an existing role
- ☐ Create a new role from AWS policy templates

Existing role

Choose an existing role that you've created to be used with this Lambda function. The role must have permission to upload logs to Amazon CloudWatch Logs.

LabRole

[View the LabRole role](#) on the IAM console.

Code:

```
import { DynamoDBClient, ScanCommand } from "@aws-sdk/client-dynamodb";
```

```
const client = new DynamoDBClient({});
```

```
export const handler = async () => {  
  const tableName = 'Ingredients';      //table name !!!important  
  const thresholdDays = 3;              //can edit value
```

```
  const today = new Date();  
  const thresholdDate = new Date(today);  
  thresholdDate.setDate(today.getDate() + thresholdDays);
```

```
  const params = {  
    TableName: tableName  
  };
```

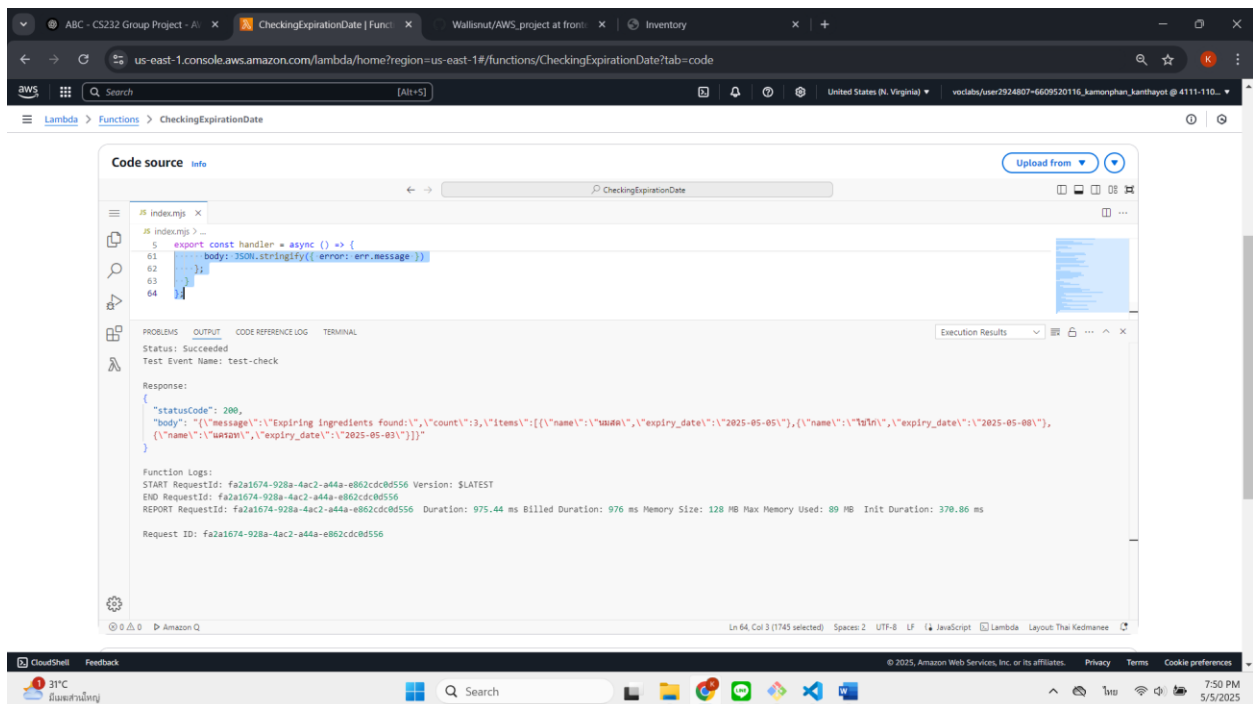
```
try {  
  const command = new ScanCommand(params);  
  const result = await client.send(command);  
  const expiringItems = [];  
  
  result.Items.forEach(item => {  
    const name = item.name.S;  
    const expiry_date = item.expiry_date.S;  
    const expiry = new Date(expiry_date);  
  
    if (expiry <= thresholdDate) {  
      expiringItems.push({ name, expiry_date });  
    }  
  });  
  
  const response = {  
    message: expiringItems.length > 0  
      ? "Expiring ingredients found:"  
      : "No expiring ingredients found.",  
    count: expiringItems.length,  
    items: expiringItems  
  };  
  
  return {  
    statusCode: 200,  
    body: JSON.stringify(response)  
  };  
} catch (err) {  
  console.error("Error scanning table:", err);  
}
```

```

return {
    statusCode: 500,
    body: JSON.stringify({ error: err.message })
};
}
};

```

***** test *****



***** ตัวอย่างผลลัพธ์ที่ Lambda จะ return *****

```
{  
  "statusCode": 200,  
  "body": "..."  
}
```

ซึ่งใน body ถ้าเราแปลง body กลับเป็น JSON (parse แล้ว):

```
{  
  "message": "Expiring ingredients found:",  
  "count": 3,  
  "items": [  
    {  
      "name": "นมสด",  
      "expiry_date": "2025-05-05"  
    },  
    {  
      "name": "ไข่ไก่",  
      "expiry_date": "2025-05-08"  
    },  
    {  
      "name": "แครอท",  
      "expiry_date": "2025-05-03"  
    }  
  ]  
}
```